Artifacts – High-Throughput Chunk-Based GPU Bitmap Compression for Dense Retrieval Workloads

José Pedro de Santana Neto[1], Simone Dominico[1], Eduardo Henrique Monteiro Pena[2], Eduardo Cunha de Almeida[1], and Marco Antonio Zanata Alves[1]

[1] Federal University of Paraná, Brazil
[2] Federal University of Technology - Paraná, Brazil

1 HiPES Data Structure Representation

Figure 1 illustrate the data structure representation of the HiPES, which is stored as a single continuous block of `uint32_t` words in memory with a fixed layout to support parallel accesses on GPUs. Each chunk block begins with a header followed by its corresponding bitmap. Each chunk occupies exactly 8,196 bytes, in which one word is used for the chunk identifier (its 16 MSB) and 2,048 words are used for its associated bitmap.

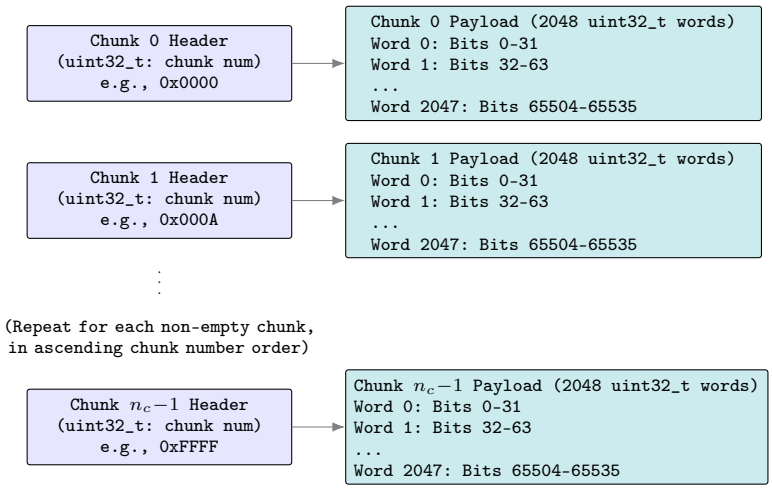


Fig. 1: Visual representation of the HiPES bitmap data structure, showing the sequence of chunk headers and their corresponding payloads. Each chunk stores its ID in a 32-bit header followed by a fixed-length bitmap encoding the 65,536-element space in a contiguous memory layout. n_c is the number of materialized (non-empty) chunks

2 Bitmap Compression Schemes

Bitmap compression research has evolved along two main paths. The first are CPU-based schemes that maximize compression ratio through variable-length encoding. Then, there are the GPU-adapted variants that target parallel throughput. Table 1 summarizes the most representative schemes, comparing their target platform, decompression model, compression ratio class, and main architectural limitations.

Tab. 1: Comparison of state-of-the-art bitmap compression schemes and the proposed HiPES scheme regarding their compression ratio, type of decompression, and the major architectural constraints.

Scheme	**Year**	**Platform**	**Decomp. Type**[a]	**Comp. Ratio**[b]	**Main Limitations**	**Ref.**
WAH	2006	CPU	*Partial*	Medium	Irregular memory access	[18]
EWAH	2008	CPU	*Partial*	Medium	Irregular memory access	[7]
Concise	2010	CPU	*Partial*	Medium	Irregular memory access	[4]
PLWAH	2010	CPU	*Partial*	High	Irregular memory access	[2]
Roaring	2016–2018	CPU	No	High	Irregular memory access and hybrid structures conflicting	[8,3]
GPU-PLWAH	2011–2021	GPU-port	*Partial*	High	Degraded throughput, reconversion overhead, warp divergence, non-coalesced memory access, global atomic contention	[10,2]
GPU-WAH	2010–2022	GPU-port	*Partial*	Medium	Warp divergence, non-coalesced memory access, global atomic contention	[11,17,1]
Falcon	2025	GPU-native	Full	High	Adaptive encoding variability, CPU-GPU transfer overhead	[9]
Blest	2025	GPU-native	N/A[c]	N/A[c]	Graph traversal focus, dependency on Tensor Cores, assumption of dense bitmaps	[5]
HiPES Scheme	**2025**	**GPU-native**	**No**	**High**	**High peak memory requirements, assumption of dense bitmaps**	**This study**

[a] Decompression Type: *Partial* indicates schemes requiring on-the-fly decoding (i.e., literal/fill handling in WAH-type formats or reconversion to WAH in PLWAH/GPU-PLWAH), which introduces overhead similar to partial decompression.
[b] Compression Ratio: Higher values indicate better space reduction relative to raw bitmaps.
[c] Blest is bitmap-centric but focuses on processing rather than compression; ratio not directly applicable.

3 GPU-WAH Implementation

The original GPU-WAH algorithm [11,17,1] uses the parallelism of SIMT to perform various operations on a pre-materialized raw bitmap. The extension phase takes each raw 32-bit word and converts it into a 31-bit WAH word. It crosses word boundaries with 64-bit temporary values and applies masks appropriately, using constants `BM31 = 0x7FFFFFFF` and `BMMSB = 0x80000000`. The phase of boundary detection identifies both where the most significant bit has been set for uniform fill words (all-0s or all-1s), and the block boundaries that exist when a literal word exists or adjacent fill words change type. In the compression phase, run lengths of identical fill blocks are encoded, producing the final compressed bitmap.

In contrast to the original GPU-WAH design, which assumes the input is already a raw bitmap, a dedicated parallel kernel `set_to_bitmap_kernel` was added in this study. This kernel aims to convert unique integer sets into bitmap format using atomic OR operations on the GPU. This conversion step is performed entirely on-device and enables a fair comparison with HiPES, which processes the integer set directly without any bitmap materialization. The measurements include the execution time of this kernel. Further optimizations are beyond the scope of the present study.

The CUDA implementation of the GPU-WAH algorithm [1] is given by Algorithm 1. It includes adaptations for architectural efficiency (i.e., Thrust). Each phase of the WAH compression process is implemented as a separate kernel, which provides high levels of parallelism within each kernel, while minimizes thread divergence and facilitates the use of coalesced memory accesses. The `exclusive_scan` function from the Thrust library is used to compute the boundaries prefix-sum step. All kernels are launched with 256 threads per block, compiled with `-O3`, and executed with GPU Boost disabled, as per the HiPES experimental conditions. The correctness is verified by a CPU-based decompression routine followed by integer set reconstruction and comparison with the input. These implementation choices ensure that both compressors operate under equal hardware, compiler, and measurement constraints.

Algorithm 1 GPU-WAH Compression Pipeline

procedure GPUWAHCompress(S)
  Input: Unordered set S of unique 32-bit integers
  Output: WAH-compressed bitmap C
  // Phase 0: Bitmap materialization
  $B \leftarrow$ raw bitmap of size $(\max(S) + 1 + 31)/32$, initialized to 0
  for $j = 0 \rightarrow |S| - 1$ **do in parallel**
    `val` $\leftarrow S[j]$
    atomicOr(B[`val`/32], $1 \ll$ (`val` mod 32))
  end for
  // Phase 1: Extension to 31-bit WAH words
  $E \leftarrow$ array of 31-bit words
  for $i = 0 \rightarrow |E| - 1$ **do in parallel**
    $E[i] \leftarrow$ extract 31 bits from B starting at bit position $31i$ // with cross-word boundary handling
  end for
  // Phase 2: Fill marking and boundary detection
  $F \leftarrow$ boundary flags
  for $i = 0 \rightarrow |E| - 1$ **do in parallel**
    if $E[i] = 0$ **or** $E[i] =$ `0x7FFFFFFF` **then**
      $E[i] \leftarrow E[i] \vee$ `0x80000000` // mark fill
    end if
    $F[i] \leftarrow 1$ if $E[i]$ is literal or fill type changes at i
  end for
  $F[|E| - 1] \leftarrow 1$ // force final boundary
  // Phase 3: Prefix sum to determine compressed size m
  $SF \leftarrow$ `exclusive_scan`(F)
  $m \leftarrow SF[|E| - 1] + F[|E| - 1]$
  // Phase 4: Build representative index array T
  for $i = 0 \rightarrow |E| - 1$ **do in parallel**
    if $F[i] = 1$ **then**
      $T[SF[i]] \leftarrow i + 1$
    end if
  end for
  // Phase 5: Final compression with run-length encoding
  $C \leftarrow$ array of size m
  for $i = 0 \rightarrow m - 1$ **do in parallel**
    $X \leftarrow E[T[i] - 1]$
    if X has MSB set **then** // fill word
      $X \leftarrow$ (fill type bits of X) $\vee$ run length of the block
    end if
    $C[i] \leftarrow X$
  end for
  return C
end procedure

4 Cross-Architecture Scalability

An important research question is to map how the algorithms behave across different hardware generations. Thus, six NVIDIA GPU architectures from Kepler (CC 3.5) to Ada Lovelace (CC 8.9) were tested. It is worth noting that the Kepler (CC 3.5) and Turing (CC 7.5) versions correspond to laptop GPU variants. A complete description of hardware characteristics is available in Table 2.

4.1 Testing Environment

The proposed tests were carried out on an idle system. Once the input data is in the host memory, the measurements start, and 100 independent repetitions were run for each setup. This approach guarantees that the statistical measures [16] are reliable and mitigates transient noise. This strategy delivers more representative measurements as a whole. All algorithms were compiled with `-O3` flag. Additionally, we deactivated GPU Boost and fixed the memory clock frequency via `nvidia-smi`. Both GPU-WAH and HiPES were implemented in CUDA [14].

Tab. 2: Characteristics of the NVIDIA GPU models used in the experiments, including their architecture family, compute capability, and release year.

GPU Model	Architecture	Compute Capability	Release Year
910M	Kepler	3.5	2015
GTX 750 Ti	Maxwell	5.0	2015
TITAN X	Pascal	6.1	2016
RTX 2060	Turing	7.5	2019
RTX A5000	Ampere	8.6	2021
RTX 6000	Ada Lovelace	8.9	2022

The list below details the specifications for each test environment:

1. **910M (2015):**
   (a) **GPU:** NVIDIA 910M, Kepler, 2 GB VRAM, Compute Capability 3.5
   (b) **CPU:** Intel(R) Core(TM) i5-5200U CPU @ 2.20GHz, 2 Cores/ 4 Threads
   (c) **System RAM:** 6 GB
   (d) **Software:** Debian (Kernel 5.4.0-150), CUDA Version 10.2, Driver 440.33.01, CUDA Compiler V10.2.89

2. **GeForce GTX 750 Ti (2015):**
   (a) **GPU:** NVIDIA GeForce GTX 750 Ti, Maxwell, 4 GB VRAM, Compute Capability 5.0, 640 CUDA Cores
   (b) **CPU:** Intel(R) Xeon(R) CPU E5620 @ 2.40GHz, 4 Cores/ 16 Threads

   (c) **System RAM:** 70 GB
   (d) **Software:** Debian (Kernel 6.1.0-37), CUDA Version 12.2, Driver 535.247.01, CUDA Compiler V11.8.89

3. **TITAN X (2016):**
   (a) **GPUs:** NVIDIA TITAN X, Pascal, 12 GB VRAM, Compute Capability 6.1, 3,584 CUDA Cores
   (b) **CPU:** Intel(R) Xeon(R) Silver 4114 CPU @ 2.20GHz, 10 Cores/20 Threads
   (c) **System RAM:** 125 GB
   (d) **Software:** Ubuntu (Kernel 6.8.0-71), CUDA Version 12.2, Driver 535.261.03, CUDA Compiler V12.5.82

4. **GeForce RTX 2060 (2019):**
   (a) **GPU:** NVIDIA GeForce RTX 2060, Turing, 6 GB VRAM, Compute Capability 7.5, 1,920 CUDA Cores
   (b) **CPU:** Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz, 6 Cores/ 12 Threads
   (c) **System RAM:** 16 GB
   (d) **Software:** Ubuntu (Kernel 6.8.0-65), CUDA Version 12.2, Driver 535.247.01, CUDA Compiler V12.0.140

5. **RTX A5000 (2021):**
   (a) **GPU:** NVIDIA RTX A5000, Ampere, 24 GB VRAM, Compute Capability 8.6, 8,192 CUDA Cores
   (b) **CPU:** Intel(R) Xeon(R) Silver 4214 CPU @ 2.20GHz, 24 Cores/ 24 Threads
   (c) **System RAM:** 125 GB
   (d) **Software:** Ubuntu (Kernel 6.8.0-71), CUDA Version 12.5, Driver 555.42.06, CUDA Compiler V12.0.140

6. **RTX 6000 (2022):**
   (a) **GPU:** NVIDIA RTX 6000 Ada Generation, Ada Lovelace, 48 GB VRAM, Compute Capability 8.9, 18,176 CUDA Cores
   (b) **CPU:** Intel(R) Xeon(R) Gold 6430, 64 Cores/ 128 Threads
   (c) **System RAM:** 500 GB
   (d) **Software:** Ubuntu (Kernel 6.8.0-71), CUDA Version 12.4, Driver 550.163.01, CUDA Compiler V12.0.140

4.2 Evaluation Metrics

A set of standard metrics was used to assess the ability of the proposed algorithm. The evaluation considers the original size of the dataset, the size of the compressed bitmap, and the time taken to perform the compression.

Compression Speed (C_{speed}) measures the throughput of the algorithm. It is calculated as the amount of initial data processed per unit of time, measured in Megabytes per second (MB/s) [15]. It is computed as in Equation 1. Where C_{speed} is the compression speed, in MB/s, S_i is the initial data size, in bytes, and T_s is the total time, taken for the whole compression process (from data in RAM to the HiPES data structure in VRAM), in seconds. The divisor 1024^2 converts bytes to megabytes.

$$C_{\text{speed}} = \frac{S_i/(1024^2)}{T_s} \tag{1}$$

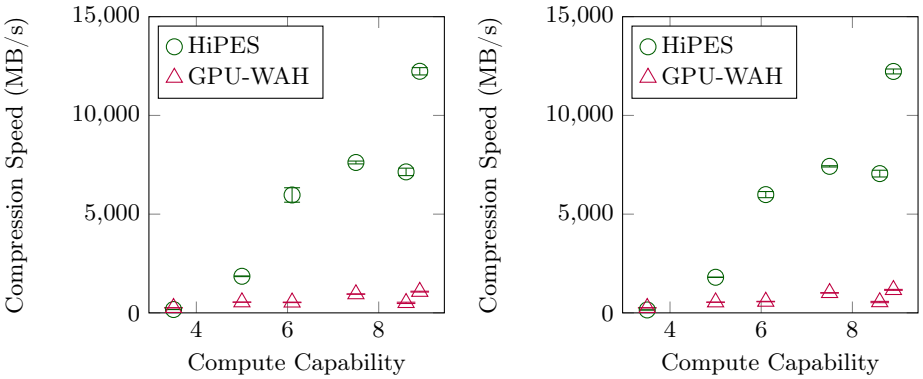
Relative Performance Ratio (RPR) compares the C_{speed} of the HiPES bitmap scheme to an existing baseline, specifically GPU-WAH. RPR features the relative computational efficiency of the approaches under analysis [6,15]. A value > 1 indicates that HiPES performs faster than its baseline in terms of compression speed. It is computed as in Equation 2. Where RPR is the Relative Performance Ratio, dimensionless, C_{speed}^{HiPES} is the compression speed of HiPES, in MB/s, and $C_{\text{speed}}^{baseline}$ is the compression speed of the baseline algorithm, in MB/s.

$$RPR = \frac{C_{\text{speed}}^{HiPES}}{C_{\text{speed}}^{baseline}} \tag{2}$$

4.3 Results

HiPES achieves a notable speed advantage over GPU-WAH, with higher compression throughput (Figure 2). Furthermore, its performance rises with GPU compute capability (CC). The throughput of HiPES peaks at about 12,230 MB/s on Ada Lovelace (CC 8.9), demonstrating that a GPU-native bitmap indexing approach can reach throughput levels comparable in order of magnitude to specialized GPU compression systems designed for other domains [9]. Both HiPES and GPU-WAH experience a performance drop on Ampere architecture (CC 8.6, RTX A5000). However, this decline is more pronounced in GPU-WAH. From Turing to Ampere, the compression speed of GPU-WAH drops by 1.88× in Scenario 2 and 1.84× in Scenario 6. On the other hand, HiPES shows a minor decline (1.07× and 1.05× for the same scenarios); thus, retaining high throughput.

We hypothesize that the decline in performance for Ampere is linked to the architectural characteristics of this generation. Ampere SMs are divided into independent sub-cores, each with its own warp scheduler, load/store unit, and a partition of the register files [12]. This allows for increased concurrency and enables Multi-Instance GPU (MIG) to operate effectively. However, there is a potential for constraints in the scheduling and resource allocation. Therefore, kernels that exhibit high levels of data dependency or use irregular memory access patterns, such as in GPU-WAH, may be subjected to highly unbalanced warp workload distributions and may have a restricted ability to utilize the available common resources.



(a) Scenario 2: Uniform Dense (b) Scenario 6: Clustered Dense

Fig. 2: Average compression speed (MB/s) of HiPES and GPU-WAH for dense workloads (Scenario 2: uniform; Scenario 6: clustered) across multiple NVIDIA GPU architectures. HiPES demonstrates increasingly superior scaling with higher compute capabilities, reflecting its GPU-native design. Error bars represent the standard deviation across multiple runs.

The average RPR is presented in Figure 3. Notably, the computed standard deviation for all samples was lower than 1%. It indicates high consistency across the measurements. Our results point to a positive trend ($R^2 = 0.78$ and $R^2 = 0.77$ for Scenarios 2 and 6, respectively) between the relative performance of HiPES and the advancements in hardware architecture. At Kepler (CC 3.5), the oldest architecture tested, HiPES underperforms GPU-WAH, with RPR values of 0.66 and 0.58 for Scenarios 2 and 6, respectively. This can be explained due to the limited shared memory atomic throughput in Kepler architecture. HiPES relies on `atomicOr` operations in shared memory for local aggregation, a strategy that is penalized by the higher atomic latency of this generation. GPU-WAH, in contrast, avoids shared memory atomics in its core phases, aligning better with these capabilities. A first performance gain of 3.4× emerges with the Maxwell architecture (CC 5.0), where shared memory atomic throughput was substantially improved. This upward trend accelerates with modern architectures, reaching performance improvements approaching one order of magnitude [13]. This trend extends to Ampere (CC 8.6) and Ada Lovelace (CC 8.9).

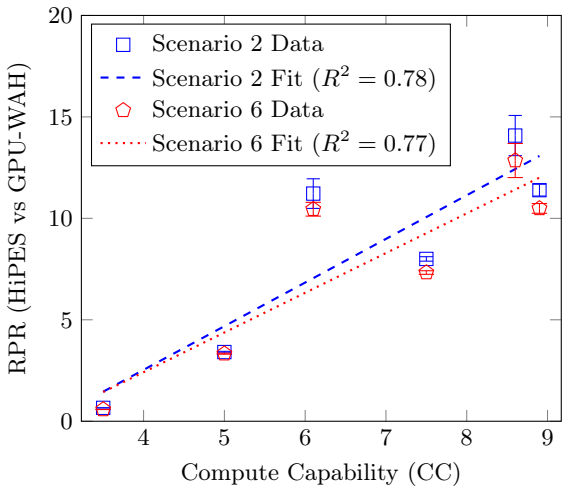


Fig. 3: Scaling behavior of HiPES relative to GPU-WAH, expressed as Relative Performance Ratio (RPR), for dense scenarios (2 and 6) across GPU architectures of increasing compute capability. Experimental averages are plotted with standard-deviation error bars, and fitted regression curves illustrate the positive correlation between compute capability and HiPES acceleration.

References

1. Andrzejewski, D., Wrembel, R.: Gpu-wah: Applying gpus to compressing bitmap indexes with word-aligned hybrid. In: 12th International Conference on Data Warehousing and Knowledge Discovery. pp. 150–163. Springer (2010)
2. Andrzejewski, D., Wrembel, R.: Gpu-plwah: Gpu-based implementation of the plwah algorithm for compressing bitmaps. In: 15th East European Conference on Advances in Databases and Information Systems. pp. 97–110. Springer (2011)
3. Chambi, S., Lemire, D., Kaser, O., Godin, R.: Better bitmap performance with roaring bitmaps. Software: Practice and Experience **46**(5), 709–719 (2016)
4. Colantonio, A., Di Pietro, R.: Concise: Compressed 'n' composable integer set. Information Processing Letters **110**(16), 644–650 (July 2010)
5. Elbek, D., Kaya, K.: Blest: Blazingly efficient bfs using tensor cores. arXiv (2025)
6. Hennessy, J.L., Patterson, D.A.: Computer Architecture: A Quantitative Approach. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 6th edn. (2017)
7. Kaser, O., Lemire, D., Aouiche, K.: Histogram-aware sorting for enhanced word-aligned compression in bitmap indexes. In: 11th International Workshop on Data Warehousing and OLAP. pp. 33–40. ACM (Sep 2008)
8. Lemire, D., Kaser, O., Kurz, N., Deri, L., O'Hara, C., Saint-Jacques, F., Ssi-Yan-Kai, G.: Roaring bitmaps: Implementation of an optimized software library. Software: Practice and Experience **48**(4), 867–895 (2018)
9. Li, Z., Wang, W., Li, R., Chen, C., Long, X., Zheng, L., Xu, Q., Yang, C.: A high-throughput gpu framework for adaptive lossless compression of floating-point data. arXiv (2025)
10. Nelson, J., Xie, T., Sun, Y.: Parallel acceleration of cpu and gpu range queries over large data sets. Future Generation Computer Systems **102**, 360–374 (2020)
11. Nelson, J., Xie, T., Sun, Y.: A statistical performance analysis of gpu wah range querying. Concurrency and Computation: Practice and Experience **34**(8), e6754 (2022)
12. NVIDIA: Nvidia ampere ga102 gpu architecture. White paper, NVIDIA Corporation (2020), `https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf`
13. NVIDIA: Nvidia ada lovelace gpu architecture. White paper, NVIDIA Corporation (2022), `https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf`
14. NVIDIA: CUDA C Programming Guide. NVIDIA Corporation (2025), release 13.0. `https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf`
15. Salomon, D., Motta, G.: Handbook of Data Compression. Springer-Verlag, London, UK, 5th edn. (2010)
16. Taylor, J.R.: An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements. University Science Books, 2 edn. (1997)
17. Tran, K., Kim, Y.K.: Exploring means to enhance the efficiency of gpu bitmap index query processing. In: 2021 IEEE International Conference on Big Data and Smart Computing. pp. 53–60 (2021)
18. Wu, K., Otoo, E.J., Shoshani, A.: Optimizing bitmap indices with efficient compression. ACM Transactions on Database Systems **31**(1), 1–38 (2006)